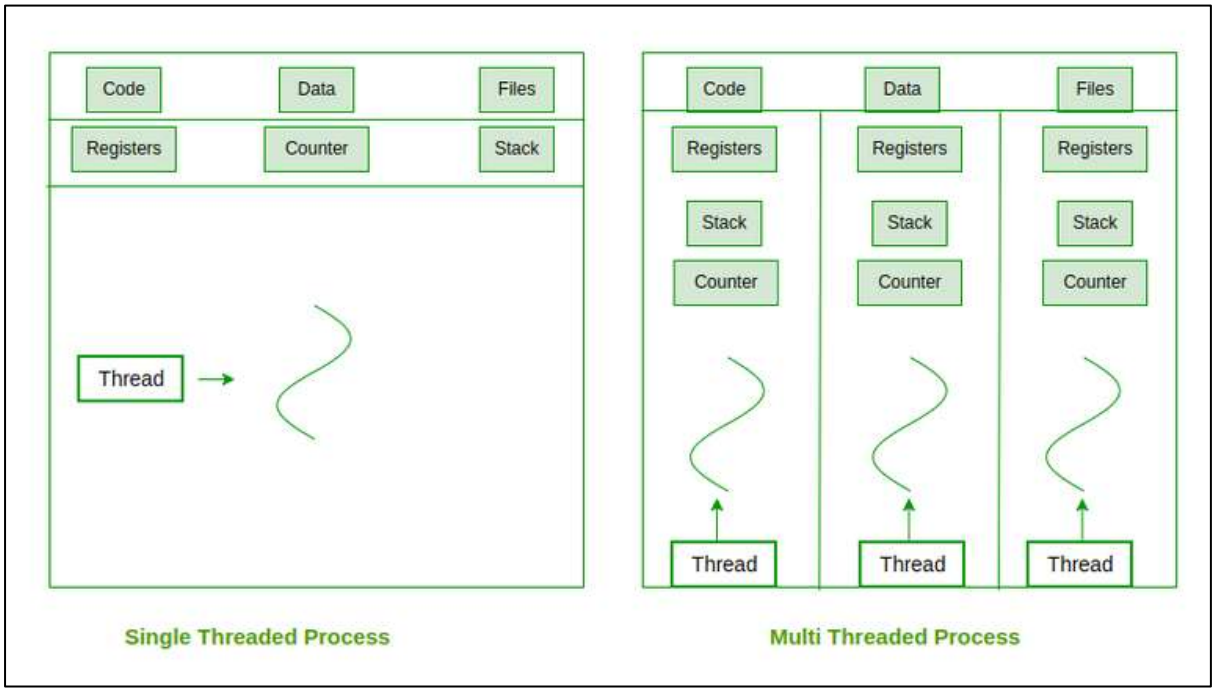
Thread

- A thread is a single sequence stream within a process.
- Threads are called **lightweight processes** as they possess some of the properties of processes. Each thread belongs to exactly one process.
- In an operating system that supports multithreading, a process can consist of many threads.
- All threads belonging to the same process share code section, data section, and OS resources (e.g. open files and signals), but each thread has its own (thread control block) - thread ID, program counter, register set, and a stack



There are two types of thread : User Thread and Kernal thread

User Thread & Kernal Thread

User-Level Threads

Managed by the application, not the operating system (Kernel). Is User-level Thread

The thread library handles creating, destroying, and managing threads.

The operating system is unaware of user-level threads.

Advantages:

- Fast to create and manage threads.
- Does not require Kernel privileges for switching threads.
- Can run on any operating system.
- Allows application-specific scheduling of threads.

Disadvantages:

- Limited Parallelism: Can't fully use multi-core processors.
- Blocking Issues: If one thread blocks (e.g., during I/O), all threads in the process are blocked.
- No True Parallelism: All user threads run on a single kernel thread.
- Not Suitable for Multiprocessor Systems: Can't use multiple CPUs effectively.

Kernel-Level Threads

Managed by the Kernel (operating system).

Kernel handles thread creation, scheduling, and management.

Threads are treated as independent entities by the operating system.

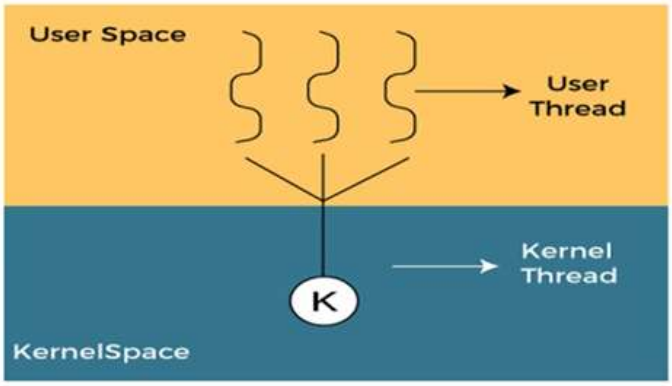
Advantages:

- True parallel execution on multi-core systems.

- If one thread blocks, the Kernel schedules another thread from the same process.
- Kernel routines can be multithreaded.

Disadvantages:

- Slower to create and manage threads due to Kernel involvement.
- Switching threads requires a mode switch to the Kernel, which adds overhead.
-



-

a)

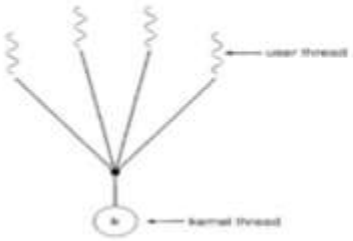
Parameter	User Thread	Kernal Thread
definition	The thread created by user in user-space is known as user thread.	The thread created by kernal in kernal space is kernal thread
Involvement	There is not involvement of O.S	There is involvement of O.S
Execution	They are faster	They are slower
Creation	Creation is easier of user thread	Creation is burden for kernal Thread
limitation	There are no limitation on User thread by O.S	There are limitation in kernal thread by O.S
Managem ent	Management is Done in user space / mode	management is done in kernel mode

Recognize	The operating System doesn't recognize user-level threads directly.	Kernel threads are recognized by Operating System.
Implementation	Implementation of User threads is easy.	Implementation of Kernel-Level thread is complicated.
Context switch time	Context switch time is less.	Context switch time is more.
Hardware support	No hardware support is required for context switching.	Hardware support is needed.
Blocking operation	If one user-level thread performs a blocking operation then the entire process will be blocked.	If one kernel thread performs a blocking operation then another thread can continue execution.
Portability	User-level threads are more portable than kernel-level threads.	Less portable due to dependence on OS-specific kernel implementations.

Multithreading model

Many systems provide support for both user and kernel threads, resulting in different multithreading models. Following are three multithreading model:

1. Many-to-One Model

- **Description**:
  Maps many user threads to one kernel thread.
  Thread management is done in user space, making it efficient.
- 

- **Key Issues**:
  - **Blocking**: If one thread makes a blocking system call, the entire process blocks.
  - **No Parallelism**: Only one thread can access the kernel at a time, so it cannot utilize multiple processors.
- **Examples**:
  Green threads in Solaris.

Green threads in Solaris.

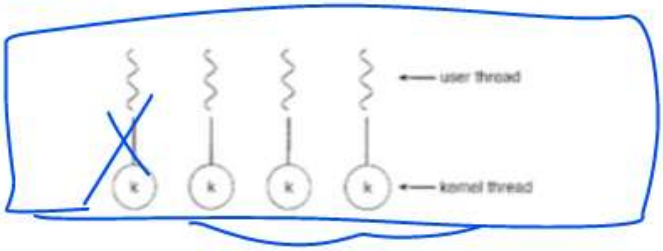
- **Advantages**:
  - Easy to implement and efficient in single-core systems.
  - Low overhead since only one kernel thread is needed.
- **Disadvantages**:
  - Blocking affects all threads.
  - No true parallelism on multiprocessor systems.

No Robust

---

2. One-to-One Model

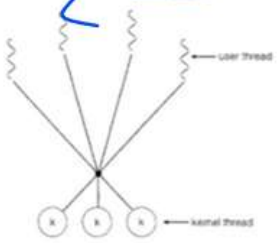
- **Description**:
  Maps each user thread to a single kernel thread.
  Allows multiple threads to run in parallel on multiprocessors.



- **Key Issues**:
  - **Overhead**: Creating a user thread requires creating a corresponding kernel thread, which can burden the system.
  - **Thread Limit**: Most systems impose limits to prevent performance issues.
- **Examples**:
  Linux and Windows operating systems.
- **Advantages**:
  - Better concurrency; other threads can run when one blocks.
  - Utilizes multiple CPUs effectively.
- **Disadvantages**:
  - High overhead due to kernel thread creation.
  - Limited number of threads can be supported.

3. Many-to-Many Model

- **Description**:
  Maps many user threads to a smaller or equal number of kernel threads.
  Developers can create many user threads, while kernel threads efficiently manage execution.



- **Key Benefits**:
  - Allows parallel execution on multiprocessor systems.
  - If a thread blocks, the kernel schedules another thread to run.
- **Advantages**:
  - Combines benefits of both Many-to-One and One-to-One models.
  - Supports true concurrency on multiprocessor systems.
  - No limit on the number of user threads.
- **Disadvantages**:
  - Overhead of managing kernel threads.
  - Complex implementation.

Benefits of Multithreading

1. **Improved Responsiveness**:
   - Allows an application to remain responsive even when performing long tasks (e.g., user interface remains active during background operations).
2. **Better Resource Utilization**:
   - Threads can share resources such as memory and data, reducing overhead compared to creating multiple processes.

---

3. **Faster Execution**:
    - Tasks can be divided among multiple threads and executed in parallel, reducing overall execution time, especially on multicore systems.
4. **Scalability**:
    - Multithreading takes advantage of multiprocessor and multicore systems by running threads simultaneously across different processors.
5. **Cost-Effective**:
    - Creating and managing threads is faster and requires fewer resources than creating processes.
6. **Enhanced Performance for I/O Operations**:
    - One thread can handle I/O operations while others continue processing, leading to better efficiency.
7. **Simplifies Complex Applications**:
    - Threads can be used to separate different functionalities of an application (e.g., one thread for input, another for computation, and another for output).
8. **Support for Asynchronous Processing**:
    - Multithreading allows programs to perform non-blocking operations, enabling tasks to proceed concurrently without waiting for others to complete.

)

ns.

Differentiate between process and thread (any two Points). Also discuss the benefits of multithreaded programming.

	Process	Thread
Definition	A process is a program under execution i.e. an active program.	A thread is a lightweight process that can be managed independently by a scheduler.
Running mechanism	Processes run in separate memory spaces.	Threads within the same process run in a shared memory space.
Concept	Process is heavy weight and any program is in execution.	It is a lightweight process and a segment of a process.
PCB	The process has its own Process Control Block, Stack, and Address Space.	Thread has Parents' PCB, its own Thread Control Block, and Stack and common Address space.
Context Switching	Context switching between the process is more expensive	Context switching between threads of the same process is less expensive
Dependency	Processes are independent.	Threads are dependent
Controlling	Process is controlled by the operating system.	Threads are controlled by programmer in a program